Project Proposal

Lab Instructor
Ma'am Noor ulAin

Submitted by:
Haroon Aziz(22i-2697)

Group Members
Haroon Aziz (I22-2697)
Khizar Shahid (i22-2595)

Date

10th April, 2026

Department of Software Engineering

FAST – National University of Computer & Emerging Sciences

MERN Stack Semester Project Proposal

---

1. Project Information

Project Title

GeoExplorer Interactive Geography Guessing Game Platform

2. Group Details

Sr. No	Name	Roll Number	Email
1	Haroon Aziz	i22-2697	i222697@nu.edu.pk
2	Khizar	i22-2595	i222595@nu.edu.pk

3. Project Overview

Problem Statement

Geography education is often limited to textbooks and static learning methods, making it less engaging for students and general users. Many learners struggle to retain information about countries, locations, and cultures because traditional methods lack interaction and visual learning.

Although geography quiz platforms exist, most are either too simple, repetitive, or lack features like progress tracking, competitive leaderboards, and real-world visual exploration.

There is a need for a modern, interactive, and engaging platform that combines visual learning, gamification, and competition to improve geography knowledge.

Proposed Solution

GeoExplorer is a full-stack MERN-based interactive geography guessing game.

The system displays random real-world location images using the Unsplash API, and users must guess the correct country based on visual clues.

Key features include:

- Score-based gameplay system
- Real-time leaderboard
- Game history tracking
- User accounts and profiles

- Admin panel for system management
- Country data integration using REST Countries API

This transforms geography learning into an engaging, competitive, and visual experience.

Target Users

- Students learning geography
- Competitive quiz/game players
- Travelers exploring destinations
- Teachers for educational activities
- Admins managing system data

4. Technology Stack

Layer	Technologies
Frontend	React.js
Backend	Node.js, Express.js
Database	MongoDB + Mongoose
Styling	Tailwind CSS
Authentication	JWT, bcrypt
HTTP Client	Axios
Maps	Leaflet.js
External APIs	Unsplash API, REST Countries API
Tools	Git, GitHub, VS Code

---

5. Figma Design Prototype (Mandatory)

User Panel Screens

1. Landing Page

- Game introduction
- Start Game button
- Instructions section

2. Login / Signup

- User authentication forms
- Validation and error handling

3. Game Screen

- Random location image
- Timer countdown
- Country selection dropdown

4. Guess Submission Screen

- Answer selection
- Submit button

5. Result Screen

- Correct answer display
- Score calculation
- Location details

6. Leaderboard

- Top players ranking
- Search and filtering options

7. User Profile

- Personal details
- Total score
- Game history

8. Game History

- Previous games list
- Filter by score/date/region

Admin Panel Screens

1. Admin Login

- Secure authentication

2. Dashboard

- Total users
- Games played
- Analytics charts

3. User Management

- View/search users
- Block/delete users

4. Game Data Management

- Manage game content
- Adjust difficulty

5. Leaderboard Control

- Reset leaderboard
- Monitor top players

6. Reports & Analytics

- Most guessed countries
- Performance stats

Navigation Flow

User Flow:

Landing Page → Login/Signup → Game Screen → Result Screen → Leaderboard / Profile → Game History

Admin Flow:

Admin Login → Dashboard → User Management / Game Management / Leaderboard / Analytics

6.USER PANEL (5 Modules)

1. Authentication Module

Description: Handles user access and identity management.

Features:

- User registration
- User login
- JWT authentication
- Password encryption (bcrypt)
- Logout functionality

2. Game Module

Description: Core gameplay module where users guess countries from images.

Features:

- Start game session
- Fetch random location images (Unsplash API)
- Display game interface
- Country guessing system
- Timer functionality
- Score calculation
- Result generation
- Feedback messages (correct/incorrect)

3. User Profile Module

Description: Manages user personal information and activity tracking.

Features:

- View profile
- Update profile
- View total score
- View games played
- Account settings

4. Game History Module

Description: Stores and manages previous gameplay records.

Features:

- Store game sessions
- View past games
- Show correct vs user guess
- Filter history (date, score, difficulty)
- Score history tracking

5. Leaderboard Module

Description: Displays ranking of players based on performance.

Features:

- Global ranking system
- Top players display
- Score aggregation
- Search players
- Filter leaderboard

ADMIN PANEL (5 Modules)

1. Admin Authentication Module

Description: Secures admin access to the system.

Features:

- Admin login
- JWT authentication
- Role-based access control
- Secure admin session

2. User Management Module

Description: Controls and manages all user accounts.

Features:

- View all users
- Search users
- Block/unblock users
- Delete user accounts
- View user statistics

3. Game Data Management Module

Description: Manages game-related system data.

Features:

- Monitor game sessions
- Manage game difficulty
- View game statistics
- Control game settings

4. Leaderboard Management Module

Description: Controls ranking system from admin side.

Features:

- Reset leaderboard
- Monitor top players
- Validate scores
- Manage ranking system

5. Analytics & Reports Module

Description: Provides system insights and performance data.

Features:

- Most guessed countries
- User activity reports
- Game performance stats
- System usage analytics
- Visual reports for admin

State Management

- Redux Toolkit:
  - authSlice authentication
  - gameSlice game state
  - leaderboardSlice rankings
- useState for UI-only state

7.API Integration Strategy

1. Centralized Axios Instance

We create a single Axios configuration file to handle all API requests.

- Sets a common base URL for backend
- Defines default headers
- Ensures consistent request handling across the app

2. JWT Interceptor for Authentication

Automatically attaches authentication token to every request.

- Retrieves JWT token from storage
- Adds token to request headers
- Ensures protected routes are accessed securely

3. Service-Based API Structure

API calls are organized into separate service files based on modules.

- authService (authentication APIs)
- gameService (game APIs)
- userService (user APIs)
- leaderboardService (ranking APIs)

This improves code organization and reusability.

4. Backend Handles External APIs

All third-party API calls are managed by the backend.

- Frontend does not call external APIs directly
- Backend handles Unsplash API and REST Countries API
- Ensures security and hides API keys
- Returns processed data to frontend

Backend Design

Description:
Defines the structure of backend APIs and how different system modules are organized on the server side.

API Architecture (Route Structure)

The backend is divided into modular API routes for better organization and scalability.

1. /api/auth

Handles authentication and user login system.

- User registration
- User login
- JWT authentication
- Get current user

2. /api/users

Handles user profile and account management.

- View profile
- Update profile
- Delete account
- User statistics

3. /api/game

Handles main game logic and gameplay operations.

- Start game session
- Submit answers
- Calculate score

- Store game results

4. /api/leaderboard

Handles ranking and competition system.

- Fetch leaderboard
- Get top players
- Rank users by score

5. /api/admin

Handles system administration features.

- Manage users
- View analytics
- Block/delete users
- Control system data

6. /api/countries

Handles country-related data.

- Fetch country list
- Get country details
- Provide country information for game logic

8. Backend Design

8.1 API Architecture

The backend follows a modular REST API structure where each major feature has its own route group:

- /api/auth - Handles authentication (register, login, logout, current user)
- /api/users - Manages user profiles and user-related operations
- /api/game - Handles game logic (start game, submit answer, score calculation, history storage)
- /api/leaderboard - Manages player rankings and score aggregation
- /api/admin - Handles administrative operations (user management, analytics, system control)

- /api/countries - Provides country data for game functionality

8.2 Authentication Strategy

The system uses secure authentication mechanisms to ensure user safety and controlled access.

- JWT (JSON Web Token) for authentication and session handling
- HTTP-only cookies to securely store tokens on the client side
- bcrypt for password hashing and encryption before storing in the database

8.3 Middleware System

Middleware functions are used to control request flow, security, and validation.

- **protect middleware** - Ensures user is authenticated before accessing protected routes
- **adminOnly middleware** - Restricts access to admin-level routes only
- **validate middleware** - Validates request data before processing
- **rateLimiter middleware** - Prevents excessive requests and protects against abuse
- **errorHandler middleware** - Handles all backend errors in a centralized way

8.4 Error Handling System

The backend uses a structured error-handling approach to ensure consistency and clarity.

- Custom AppError class for generating standardized errors
- Central error-handling middleware to catch and process all errors
- Standardized API response format for both success and error responses

9. API Endpoints

Authentication

Method	Endpoint	Description
POST	/api/auth/register	Register user
POST	/api/auth/login	Login user

GET	/api/auth/me	Get user info

Game

Method	Endpoint	Description
GET	/api/game/start	Start game
POST	/api/game/submit	Submit answer
GET	/api/game/history	Game history

Leaderboard

Method	Endpoint
GET	/api/leaderboard
GET	/api/leaderboard/top10

Countries

Method	Endpoint
GET	/api/countries
GET	/api/countries/:code

10. Database Design

Collections

- Users
- GameSessions
- Leaderboard

Users

Field	Type
name	String
email	String
password	String
role	String
totalScore	Number
gamesPlayed	Number

GameSessions

Field	Type
userId	ObjectId
imageUrl	String
correctCountry	String
userGuess	String
score	Number
difficulty	String

Leaderboard

Field	Type
userId	ObjectId
totalScore	Number
rank	Number

11. CRUD Operations Mapping

The system follows complete CRUD operations across different modules including Users, Game Sessions, Leaderboard, and Admin controls.

1. User Module CRUD

Operation	Description	API Endpoint
Create	Register new user account	POST /api/auth/register
Read	Get user profile and details	GET /api/users/profile
Update	Update user profile information	PUT /api/users/update
Delete	Delete user account	DELETE /api/users/:id

2. Game Module CRUD

Operation	Description	API Endpoint
Create	Start a new game session	GET /api/game/start
Read	View game session details	GET /api/game/history

Update	Update game result (score submission)	POST /api/game/submit
Delete	Remove invalid game session (admin use)	DELETE /api/game/:id

3. Game History CRUD

Operation	Description	API Endpoint
Create	Store completed game session	Auto-saved after game ends
Read	Fetch user game history	GET /api/game/history
Update	Update game record (score correction if needed)	PUT /api/game/history/:id
Delete	Delete old game records (admin only)	DELETE /api/game/history/:id

4. Leaderboard CRUD

Operation	Description	API Endpoint
Create	Add new user score entry	Auto-generated after game completion

Read	View leaderboard rankings	GET /api/leaderboard
Update	Update user total score after each game	POST /api/leaderboard/update
Delete	Reset leaderboard (admin control)	DELETE /api/leaderboard/reset

5. Admin Module CRUD

Operation	Description	API Endpoint
Create	Create admin account (initial setup)	POST /api/auth/admin/register
Read	View system data (users, games, stats)	GET /api/admin/dashboard
Update	Modify user/game/system settings	PUT /api/admin/update
Delete	Remove users or reset system data	DELETE /api/admin/:id

12. Search and Filtering

Search

- Player name
- Country name
- Email

Filters

- Region
- Score range
- Difficulty
- Date range

13. External APIs

The system integrates external APIs to provide real-world data and improve user experience in the game.

Unsplash API

- A free image service used to fetch high-quality real-world images
- Requires an API key for authentication
- Used to display random location-based images in the game
- Helps in visually representing countries or places for guessing

Usage in project:

- Game Module uses Unsplash API to fetch random location images
- Backend requests images and sends them to frontend

REST Countries API

- A free public API that provides detailed information about countries
- Does not require any API key
- Provides data such as country names, flags, codes, regions, and coordinates

Usage in project:

- Used to validate and map user guesses
- Provides correct country information for game logic
- Helps in generating hints and verifying answers

14. Chatbot Integration (Optional Feature)

Gemini API (Google AI)

- Used to integrate an AI-based geography assistant chatbot
- Allows users to ask geography-related questions
- Provides intelligent responses about countries, capitals, and locations

Features:

- Answer geography questions
- Provide hints during gameplay
- Assist users in learning countries interactively

Purpose in project:

- Enhances learning experience
- Adds AI-based interaction to the platform
- Optional feature to improve engagement

15. System Architecture Summary

The system follows a **full-stack MERN architecture** with clear separation between frontend, backend, database, and external services.

Architecture Flow:

1. Frontend (React.js)

- Handles user interface
- Displays game, leaderboard, profiles, and admin panel
- Sends requests to backend APIs

2. State Management (Redux)

- Manages global application state
- Stores:
    - User authentication state
    - Game state
    - Leaderboard data

- Ensures consistent data flow across components

3. Backend (Node.js + Express.js)

- Handles business logic
- Processes API requests
- Manages authentication and authorization
- Communicates with database and external APIs

4. Database (MongoDB)

- Stores all persistent data
- Collections include:
    - Users
    - GameSessions
    - Leaderboard

5. External APIs

- Unsplash API → Image data
- REST Countries API → Country information
- Gemini API → AI chatbot (optional)

Overall Flow:

Frontend → Backend → Database / External APIs → Backend → Frontend

16. ER Diagram Summary

The system database follows relational logic using MongoDB collections.

1. Users → GameSessions (One-to-Many Relationship)

- One user can play multiple games
- Each game session belongs to one user
- Stores game history, score, and guesses

2. Users → Leaderboard (One-to-One Relationship)

- Each user has one leaderboard entry
- Stores total score and rank

- Updated after each game

Key Entities:

- Users
- GameSessions
- Leaderboard
-

17. Folder Structure

To keep the project organized and easy to maintain, the system is divided into **frontend** and **backend** folders. Each part of the project contains files grouped according to their functionality.

Frontend Structure (React)

The frontend is developed using **React.js**, and the main code is located inside the src folder. This folder is divided into several subfolders that handle different parts of the user interface.

components
This folder contains reusable UI components used across the application. The components are further organized based on features such as common UI elements, game-related components, leaderboard interface, user profile components, and admin panel components.

pages
This folder contains the main pages of the application that users navigate through. These include the Home page, Game screen, Leaderboard page, User Profile page, and the Admin Dashboard.

redux
Redux Toolkit is used for global state management. This folder contains slices that manage authentication, game state, and leaderboard data.

services
The services folder contains files responsible for handling API requests between the frontend and backend. These services manage operations related to authentication, game functionality, users, and leaderboard data.

utils
Utility functions and helper files are placed in this folder. It includes the Axios instance used for API communication and helper functions used throughout the project.

Backend Structure (Node.js + Express)

The backend of the system is built using **Node.js and Express.js** and follows a modular structure similar to the MVC pattern. This structure helps separate application logic, routing, and data models.

routes
This folder defines all API endpoints of the system. Each file handles routes related to a specific feature such as authentication, users, game operations, leaderboard management, admin functionality, and country data.

controllers
Controllers contain the main business logic of the application. They process incoming requests, perform required operations, and send responses back to the client.

models
This folder contains MongoDB schemas created using Mongoose. The main models include the User model, GameSession model, and Leaderboard model.

middleware
Middleware functions are used for tasks such as authentication, authorization, request validation, error handling, and rate limiting.

config
Configuration files are stored in this folder. It includes the database connection setup and other environment-related configurations.

server.js
This is the main entry point of the backend application where the Express server is initialized and all routes and middleware are connected.

18. Work Distribution (Panel-wise Division)

The system is divided into two main panels:

- User Panel
- Admin Panel

The project consists of 10 modules in total. Both members contributed as full-stack developers across both frontend and backend. Each member is assigned an equal number of modules with clear responsibilities.

Haroon Aziz (I22-2697)

User Panel Responsibilities

1. Authentication Module (Full Stack)

Frontend:

- Login page UI
- Signup page UI
- Form validation and error handling

Backend:

- JWT authentication system
- bcrypt password encryption
- `/api/auth` routes
- protect middleware for secured routes

2. Game Module (Core Module – Full Stack)

Frontend:

- Game interface design
- Image display screen (Unsplash images)
- Country selection/guessing system
- Timer and submission UI

Backend:

- Game session creation and handling
- Score calculation logic
- Result evaluation system
- `/api/game` routes
- Integration with Unsplash API and REST Countries API

3. Game History Module (Full Stack)

Frontend:

- Game history UI
- Filters (date, score, difficulty)
- Display of past game sessions

Backend:

- Store game session data in MongoDB
- Retrieve user game history
- Filter history based on conditions
- History-related APIs

4. Leaderboard Module (Full Stack)

Frontend:

- Leaderboard UI
- Top players display
- Search and filter functionality

Backend:

- Ranking system based on scores
- Score aggregation logic
- `/api/leaderboard` routes
- Real-time leaderboard updates logic

5. Part of User Profile Module (Full Stack)

Frontend:

- User profile UI
- Display total score and game stats

Backend:

- Fetch user details
- Update user statistics
- Manage profile-related data

Additional Responsibilities (Haroon)

- Axios centralized instance setup
- JWT interceptor implementation
- Redux state management (auth + game)
- API service structure for modules
- GitHub repository setup and version control
- Frontend-backend integration testing

Khizar Shahid (I22-2595)

Admin Panel Responsibilities

1. Admin Authentication Module (Full Stack)

Frontend:

- Admin login UI
- Secure login form

Backend:

- Admin authentication system
- Role-based access control
- `/api/auth/admin` handling
- adminOnly middleware

2. User Management Module (Full Stack)

Frontend:

- User management dashboard
- View users interface
- Search users UI

Backend:

- User CRUD operations
- `/api/users` routes
- Block/unblock users
- Delete user accounts
- User statistics tracking

3. Game Data Management Module (Full Stack)

Frontend:

- Game data control interface
- Admin controls for game settings

Backend:

- Manage game session data
- Adjust difficulty levels
- Monitor game activity
- Game management APIs

4. Leaderboard Management Module (Full Stack)

Frontend:

- Leaderboard control UI
- Search/filter leaderboard

Backend:

- Ranking validation
- Score verification system
- `/api/leaderboard` admin controls
- Reset/update leaderboard functionality

5. Analytics & Reporting Module (Full Stack)

Frontend:

- Admin analytics dashboard
- Charts and statistics UI

Backend:

- System analytics generation
- Most guessed countries data
- User activity tracking
- Performance reports

Additional Responsibilities (Khizar)

- MongoDB schema design (Users, GameSessions, Leaderboard)
- Backend architecture setup (Express structure)
- Input validation middleware
- Error handling system implementation
- Rate limiter middleware
- API response standardization
- Deployment setup and configu

19. References

- Unsplash API
- REST Countries API
- React.js
- Node.js
- MongoDB
- Tailwind CSS
- Redux Toolkit